

GRADO EN TECNOLOGÍAS DIGITALES PARA LA EMPRESA

Universidad de Burgos

ASIGNATURA

Programación Orientada a Objetos (POO)

TRABAJO / PRÁCTICA

Práctica I: Planta Industrial Simulada

Estudiante	Fernando Miguel Morán Fernández
DNI / NIE	71378005N
Correo electrónico	fmf1007@alu.ubu.es
Grupo	Grupo 1
Profesor/a	Jaime Andrés Rincón Aragón
Curso académico	2026–2027
Fecha de entrega	4 de noviembre de 2026

Campus de Miranda de Ebro **Universidad de Burgos**

Índice

1. Introducción **3**

2. Objetivos **3**

2.1. Objetivo general 3

2.2. Objetivos específicos 3

3. Análisis del problema **4**

3.1. Requisitos funcionales 4

3.2. Restricciones y consideraciones 4

4. Diseño orientado a objetos **4**

4.1. Identificación de clases 5

4.2. Diagrama de clases UML 5

4.3. Relaciones entre clases 6

5. Principios de POO aplicados **6**

5.1. Encapsulación 6

5.2. Abstracción 6

6. Implementación **6**

6.1. Estructura del proyecto 6

6.2. Decisiones de diseño: Visión Artificial 7

7. Manejo de errores y validaciones **7**

8. Pruebas y validación **8**

9. Resultados **8**

10. Discusión **9**

11. Conclusiones **9**

12.Contribución individual **9**

13.Declaración sobre el uso de herramientas de inteligencia artificial **9**

14.Referencias **9**

A. Código fuente y repositorio **11**

1. Introducción

El presente trabajo aborda la migración y refactorización de un simulador de planta industrial, basado originalmente en el paradigma de programación procedural, hacia una arquitectura de Programación Orientada a Objetos (POO). Este simulador físico, desarrollado utilizando el motor gráfico **MuJoCo** y la librería de visión artificial **OpenCV** en el lenguaje Python, gestiona un entorno automatizado compuesto por cinco cintas transportadoras, un brazo robótico Franka Panda y un horno industrial.

- **Contexto y motivación:** El código original dependía de variables globales (flags) y listas de estados genéricas, lo que dificultaba la escalabilidad y mantenibilidad del sistema. La POO permite modularizar cada componente físico de la fábrica.
- **Funcionalidades principales:** Interacción física de piezas sobre cintas, detección de formas geométricas en tiempo real (visión artificial) y alteración dinámica del entorno (horno).
- **Lenguaje y herramientas:** Python, MuJoCo (simulación física) y OpenCV (visión por computador).

2. Objetivos

2.1. Objetivo general

Diseñar e implementar una estructura puramente orientada a objetos para el control de la planta industrial simulada, eliminando el estado global y encapsulando el comportamiento de actuadores y sensores en sus respectivas clases.

2.2. Objetivos específicos

1. Diseñar las clases `PlantaIndustrial`, `CintaTransportadora`, `CamaraInspeccion` y `Horno`.
2. Implementar relaciones de composición entre la planta, las cámaras y las cintas.
3. Desarrollar un sistema de visión artificial integrado en la clase cámara capaz de detener su cinta asociada al detectar y clasificar una pieza (Cilindro o Caja).
4. Modificar dinámicamente las propiedades de los materiales en MuJoCo (color del horno) basándose en los estados internos del objeto (temperatura).

3. Análisis del problema

El simulador presentaba una dependencia crítica de funciones centralizadas. Para el diseño orientado a objetos, se identificaron los siguientes requerimientos y restricciones.

3.1. Requisitos funcionales

1. **Detección y clasificación:** Las cámaras deben procesar la imagen RGB, convertirla a escala de grises, umbralizarla y utilizar aproximación poligonal para distinguir cilindros (> 6 vértices) de cajas.
2. **Control de flujo automático:** Cuando una pieza ingresa en la zona de detección central (rectángulo azul) de una cámara, la cinta transportadora correspondiente debe detenerse automáticamente para evitar caídas o atascos.
3. **Simulación térmica:** El horno industrial debe contar las piezas procesadas y cambiar el color de su geometría de manera dinámica (Azul < 50°C, Naranja < 100°C, Rojo > 100°C) en respuesta a comandos externos.

3.2. Restricciones y consideraciones

Se prohibió estrictamente el uso de variables globales. La comunicación entre el reconocimiento visual (cámara) y la física del motor (cinta) debe realizarse exclusivamente a través de referencias a objetos pasadas por parámetros en los constructores. Adicionalmente, se debieron ajustar los parámetros de umbralización en OpenCV para lidiar con el texturizado de los materiales renderizados por MuJoCo.

4. Diseño orientado a objetos

4.1. Identificación de clases

Cuadro 1: Clases principales de la aplicación.

Clase	Responsabilidad	Elementos principales
PlantaIndustrial	Coordinar el simulador, renderizar cámaras y gestionar el bucle físico.	`self.cintas`, `self.camaras`, `self.horno`.
CintaTransportadora	Actuar sobre las fuerzas de MuJoCo para mover las piezas.	`conmutar()`, `detener()`, `transport_force()`.
CamaraInspeccion	Sensor virtual de visión artificial. Manda órdenes de parada.	`capturar()`, `detectar_forma()`, `actualizar()`.
ObjetosIndustrial	Gestionar las piezas, su transporte, reinicio y estabilidad física.	`get_ids()`, `update_segment()`, `apply_transport()`.
Horno	Gestionar temperatura, conteo de piezas y apariencia visual.	`registrar_pieza()`, `actualizar_color()`.

4.2. Diagrama de clases UML

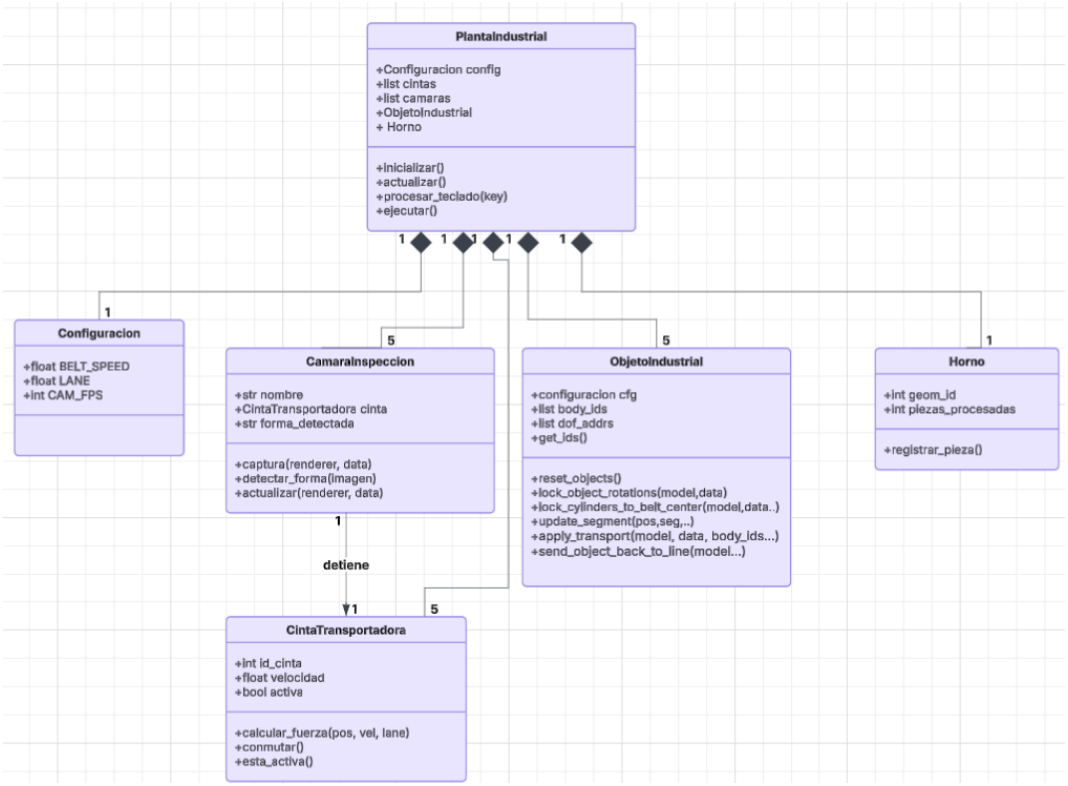


Figura 1: Diagrama de clases UML de la Planta Industrial.

4.3. Relaciones entre clases

- **Composición:** La clase principal PlantaIndustrial instancia y contiene listas de objetos CintaTransportadora y CamaraInspeccion, así como un Horno. Si la planta se destruye, sus componentes también.
- **Asociación unidireccional:** Cada objeto CamaraInspeccion guarda una referencia a una CintaTransportadora. Esto permite que el sensor visual interactúe directamente con su actuador asociado.

5. Principios de POO aplicados

5.1. Encapsulación

El estado interno de activación de las cintas (ON/OFF) y la temperatura del horno están protegidos. El bucle de ejecución principal no modifica la velocidad de las cintas directamente en los arrays de MuJoCo, sino que invoca métodos semánticos como **self.cintas[0].detener()** o **self.horno.aumentar_temperatura(10.0)**.

5.2. Abstracción

La complejidad de OpenCV (extracción de contornos, cálculo de momentos espaciales, aproximación poligonal) queda completamente oculta dentro de la clase **CamaraInspeccion**. El controlador principal simplemente invoca **cam.actualizar(renderer, data)**, abstrayéndose del proceso de filtrado de píxeles.

6. Implementación

6.1. Estructura del proyecto

El código se ha modularizado separando cada entidad física y de configuración en su propio archivo dentro del proyecto:

- **main.py**: Punto de entrada principal de la aplicación situado en la raíz.
- **src/planta.py**: Clase controladora principal (`PlantaIndustrial`).
- **src/configuracion.py**: Clase `Configuracion` con los parámetros geométricos y físicos del entorno.
- **src/cinta.py**: Lógica de tramos y cálculo de fuerzas de transporte (`CintaTransportadora`).

- **src/camara.py**: Módulo de visión artificial, captura de fotogramas y clasificación con OpenCV (`CamaraInspeccion`).
- **src/objetos.py**: Clase `ObjetosIndustrial` para el control cinemático, estados y restablecimiento de las piezas.
- **src/horno.py**: Gestión térmica, conteo y renderizado dinámico del horno industrial.
- **src/robot.py**: Módulo de control del brazo robótico Franka Panda.

6.2. Decisiones de diseño: Visión Artificial

Para asegurar la correcta detección en MuJoCo, se implementaron soluciones específicas en OpenCV:

1. **Ajuste poligonal:** Se redujo el factor de aproximación perimetral a 0.01 * perimetro para evitar que las formas circulares (cilindros) fuesen simplificadas en polígonos de menos de 6 vértices.
2. **Zona delimitada de activación:** Se programó una zona muerta mediante el cálculo de momentos espaciales de los contornos detectados. La parada automática de la cinta se activa exclusivamente si la coordenada central de la pieza cruza los márgenes de una caja lógica virtual proyectada en el centro de la lente.

7. Manejo de errores y validaciones

Durante el desarrollo se gestionaron diversos casos límite críticos:

- **Filtrado de ruido visual:** La función de umbralización térmica se afinó mediante el ajuste paramétrico de áreas poligonales, descartando artefactos menores a 50 píxeles y el fondo perimetral de la propia cinta (> 12000 px), evitando así falsos positivos en las lecturas de los sensores ópticos.
- **Modificación de atributos precompilados:** Para posibilitar la actualización del vector cromático (RGBA) del Horno, se desvinculó de su geometría en el archivo XML el atributo estático material, concediendo a la clase control absoluto sobre el motor de renderizado.

8. Pruebas y validación

Cuadro 2: Plan de pruebas funcionales.

ID	Entrada / Escenario	Resultado esperado	Estado
P01	La pieza viaja en la Cinta 1 e ingresa en el campo de visión de la cámara.	La cámara detecta la pieza, procesa la forma y la cinta 1 se detiene automáticamente.	Correcto
P02	Alteración manual del termostato del horno mediante teclado ('m,+'/'n,-').	El vector RGBA del objeto geom en MuJoCo se actualiza (Azul → Rojo).	Correcto
P03	Discriminación geométrica entre Caja (amarilla) y Cilindro (verde).	OpenCV aproxima polígonos. Cilindros devuelven > 6 vértices. Cajas devuelven ≤ 6.	Correcto

9. Resultados

Tras la refactorización, el simulador funciona de manera fluida demostrando la comunicación eficiente entre las instancias. El mosaico de visión artificial proyecta en tiempo real los bounding boxes poligonales sobre las piezas y clasifica su morfología visualizada en pantalla, logrando la parada instantánea del rodillo simulado.

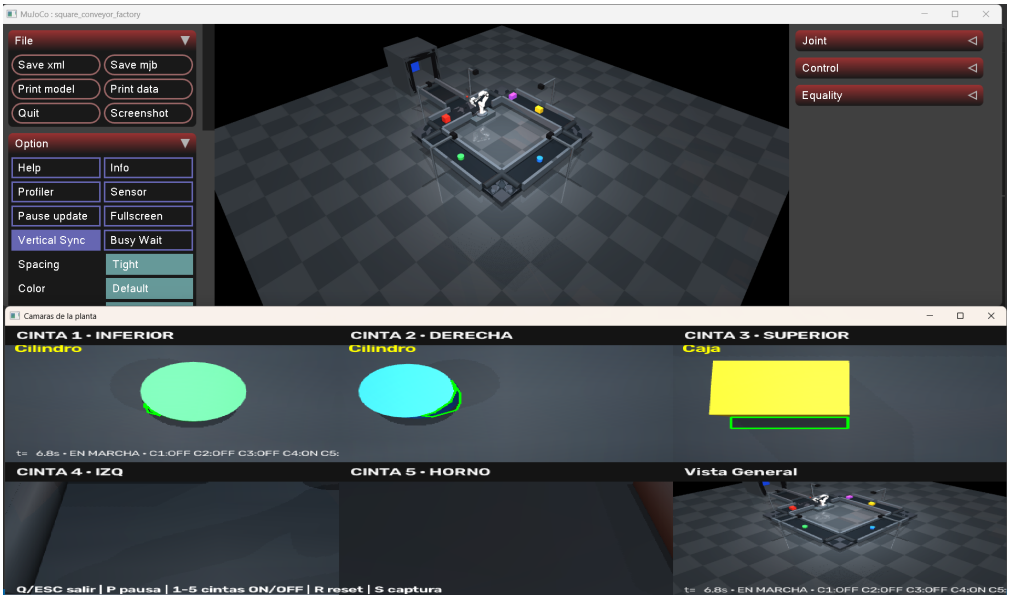


Figura 2: Sistema POO de automatización con clasificación visual (Cajas y Cilindros).

10. Discusión

El diseño propuesto soluciona los problemas de alta dependencia (acoplamiento) iniciales.

- **Ventajas:** Al instanciar las cintas transportadoras de manera independiente, añadir una sexta cinta a la planta requiere ahora únicamente añadir una línea de código en el constructor de la planta general.
- **Mantenibilidad:** El algoritmo de contornos se ha aislado. Si en el futuro se desea cambiar OpenCV por un modelo de Machine Learning para detección de objetos, solo se modificará el método detectar_forma() de la clase CamaraInspeccion, sin afectar en absoluto a la lógica de transporte de MuJoCo.

11. Conclusiones

Se ha logrado el rediseño total de la planta de control, alcanzando los objetivos especificados en la práctica. La refactorización ha demostrado las ventajas de la programción Orientado a Objetos para la robótica y la simulación física.

12. Contribución individual

Cuadro 3: Contribución de los integrantes del grupo.

Estudiante	**Contribución**
Fernando Morán	Práctica completa.

13. Declaración sobre el uso de herramientas de inteligencia artificial

Se utilizó una herramienta de inteligencia artificial de apoyo para resolver dudas técnicas (por ejemplo, en el informe en latex como comados y ajuste de imagen, además de consultas sobre funciones específicas de OpenCV y Mujuco).

14. Referencias

Referencias

[1] DeepMind, *MuJoCo: A physics engine for model-based control*. Documentation. https://mujoco.org/

[2] OpenCV, *Structural Analysis and Shape Descriptors: approxPolyDP, findContours*. https://docs.opencv.org/4.x/

A. Código fuente y repositorio

El código fuente completo de esta práctica ha sido subido a un repositorio para facilitar su compilación y revisión:

Repositorio GitHub: https://github.com/Fer851/planta_insdustrial

Instrucciones de ejecución: Para ejecutar la simulación, iniciar un entorno virtual, instalar los requerimientos (`pip install -r requirements.txt`) y lanzar el archivo principal posicionado en la raíz:

```
python main.py
```